Cheatsheet: Creating a Quarto Website

From installation to a published website

OUTPUT	STRUCTURE	PUBLISH
Full multi-page site (HTML)	One project, many `.qmd` pages	GitHub Pages (`docs/` folder)

1 — Setup (once per computer)

Install:

- **VS Code** — code.visualstudio.com (tick **"Add to PATH"** during setup)
- **Quarto CLI** — quarto.org/docs/get-started
- **Quarto extension** for VS Code — search "Quarto" in the Extensions sidebar

Create the project:

File → New File... → Quarto Project → Website Project → choose your folder. Quarto creates:

- `_quarto.yml` — project-wide settings (see section 2)
- `index.qmd` — your homepage
- `about.qmd` — an example second page
- `styles.css` — optional custom styling

Ready to start editing? See **Preview vs. Render** at the end of section 2 before you do.

Which one do I need?

Building something with several linked pages and site-wide navigation — a course site, documentation, or portfolio? This is it. For one self-contained document to print or hand out, see the **Exercise Sheet** cheatsheet; for a long, sequentially-read document with numbered chapters, see the **Book** cheatsheet.

2 — Writing & editing content

Every `.qmd` file is a **YAML header** followed by a **Markdown body**. There are two separate, independent levels of YAML — don't confuse them:

1. **Project-wide, one file:** `_quarto.yml` sets navigation, theme, and defaults for the *entire* site. It exists once per project — you only touch it for site-wide changes, not for every page.
2. **Per page, every file:** the `---`-fenced header at the *top of each* `.qmd` file. Every page has one; it sets that page's own title and can override the project's defaults just for itself.

Project-wide (`_quarto.yml`):

```yaml
website:
  title: "My Site"
  navbar:
    left:
      - text: "About"
        href: about.qmd
```

Per page (top of `my-page.qmd`):

```yaml
---
title: "My Page"
format: html
filters:
  - parmsam/quizdown
---
```

Markdown body basics:

Syntax	Result
`## Heading`	Section heading
`**bold**`, `*italic*`	**bold**, *italic*
`` `code` ``	inline code
`- item` / `1. item`	Bulleted / numbered list
`> text`	Blockquote
`[text](url)`	Link
`![](path.png)`	Image
`$x^2$`, `$$x^2$$`	Inline / block math
fence tagged `{python}`	Executable code cell (see below)
`::: {.callout-tip} ... :::`	Callout box

Callouts flag information visually:

```
::: {.callout-tip}
## Tip
Use tips for best-practice advice.
:::
```

renders as:

> **Tip**
>
> Use tips for best-practice advice.

Other types: `.callout-note`, `.callout-warning`, `.callout-important`.

Code cells: open a fence tagged `{python}` (or `{pyodide-python}` for interactive) and close it with a plain fence, e.g.:

```
#| echo: false   # hide the code, keep the output
#| eval: true    # run the cell (default)
print("Hello!")
```

Common cell options (as `#|` comments): `echo`, `eval`, `warning`, `error`, `output`.

Preview vs. Render:

Command	What it does	Use it when
Preview button, or `quarto preview`	Opens a live local view; updates automatically every time you save a file	While you're writing and editing
`quarto render`	Builds the final static output files into `_site` (or `docs/`, if configured) — no live view opens	Before publishing, or to produce the files to upload

> **Tip**
>
> The **Preview** button does both at once: it renders your site and opens the live view. You only need a separate `quarto render` when you want the output files *without* opening a preview — e.g. right before publishing.

3 — Publish with GitHub Pages

1. In `_quarto.yml`, under `project:`, add `output-dir: docs`
2. Render: `quarto render`
3. Create a **public** repository on github.com
4. Upload the whole project folder (including `docs/`) and commit
5. Repo → **Settings** → **Pages** → Branch `main`, folder `/docs` → **Save**
6. Wait for the green checkmark under **Deployments** — your site is live at `https://<username>.github.io/<repo-name>/`

Updating your site later: run `quarto render` again, then re-upload the `docs/` folder to GitHub (or `git push` if you're using Git).

Good to know

- `site-url:` in `_quarto.yml` sets the canonical base URL — needed for correct social-share previews and RSS links.
- `repo-url:` and `page-footer:` add an automatic "Edit this page" / source-code link and a custom footer, site-wide.
- Renaming or moving a `.qmd` file changes its output URL. Plan file names before publishing — old links to a renamed page will 404.

Full details & screenshots: What you need · Create a Website · Publish a Website